

Next.js Lesson 5.1 — Forms + Server Actions 🔥

Yep — picking up exactly where we left off.

We established:

Route Handler → HTTP door 🚪
Server Action → server-function door 🚪
BFF → architectural role 🏗️



Now let's connect that to the **thing that actually triggers mutations: forms and user interactions.**

5.1.1 — The basic problem

Imagine our Performance Analyzer has a form:

Enter URL
↓
[https://example.com]
↓
[Analyze]



When the user clicks **Analyze**, we need to:

1. Get the URL from the form
2. Send it to the server
3. Call Google PageSpeed API from the server
4. Process the result
5. Return/update the UI

In traditional React, you might think:

```
Browser
  ↓
onSubmit()
  ↓
fetch("/api/analyze")
  ↓
Route Handler
  ↓
PageSpeed API
```



With **Server Actions**, Next.js gives us another path:

```
Browser
  ↓
<form action={serverAction}>
  ↓
Server Action
  ↓
PageSpeed API
```



And importantly:

the browser does NOT receive your server-side function code.

5.1.2 — A Server Action can be used directly by a form

Example:

```
// actions.ts

"use server";

export async function analyzeWebsite(formData: FormData) {
  const url = formData.get("url");

  console.log(url);
}
```

```
// Call PageSpeed API here  
}
```

Then:

```
// page.tsx  
  
import { analyzeWebsite } from "../actions";  
  
export default function Page() {  
  return (  
    <form action={analyzeWebsite}>  
      <input name="url" />  
      <button type="submit">  
        Analyze  
      </button>  
    </form>  
  );  
}
```

Look carefully at this:

```
<form action={analyzeWebsite}>
```

We aren't doing:

```
onSubmit={() => {  
  fetch(...)  
}}
```

We're literally telling React/Next:

"When this form is submitted, execute this server action."

That's the cool part. 😄

5.1.3 — Where does `formData` come from?

This:

```
<input name="url" />
```

produces a form field.

When submitted:

```
<input name="url" value="https://google.com">
```

the server action receives:

```
formData
```

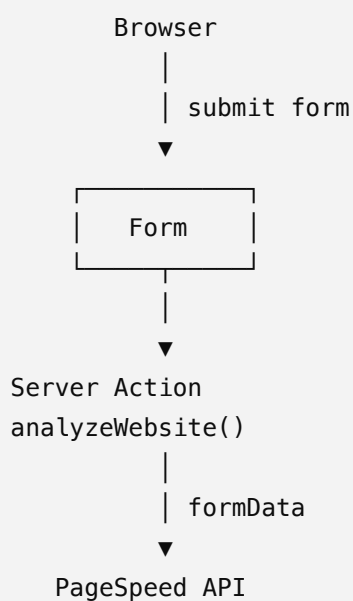
So:

```
const url = formData.get("url");
```

gives:

```
"https://google.com"
```

The flow is:



5.1.4 — Why is this called a "mutation"?

Because we're **causing something to happen**, rather than simply reading data.

Reading

"Give me the performance results."



That's a **query/read**.

Mutation

"Analyze this URL."
"Create this account."
"Update this profile."
"Delete this item."
"Submit this form."



These are **actions that cause something to happen**.

So Server Actions are particularly useful for:

User interaction
↓
Mutation
↓
Server-side work



5.1.5 — Here's where your BFF mental model matters

Remember our earlier discussion.

A BFF is an **architectural role**, not necessarily a particular Next.js feature.

So we can have:

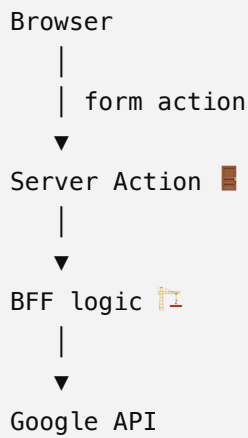
Option A — Route Handler

Browser
|
| HTTP
▼
Route Handler 
|
▼
BFF logic 
|



Google API

Option B — Server Action



That's why the distinction we made earlier is useful:

Route Handler and Server Action are different doors into server-side code.

And:

BFF describes what that server-side layer is doing architecturally.



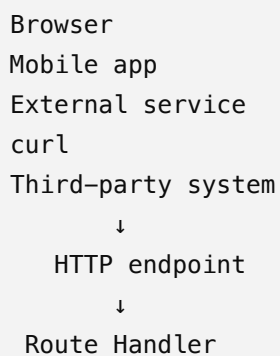
That's the mental model I want you to keep.

One important difference

A Route Handler is fundamentally an **HTTP endpoint**:

POST /api/analyze

Therefore another client can call it:



A Server Action is primarily designed around **your application's server functions and UI interactions**:

Your React/Next UI



Server Action



Server-side code



So don't think:

"Server Actions replace Route Handlers."

Instead think:

"They solve overlapping but different communication problems."

And this leads directly to the next concept 📌

Now that we know a form can directly trigger a Server Action, the next question is:

What if I need the UI to know whether the action is loading, succeeded, failed, or returned some result?

That's where we'll go next:

`useActionState` → pending state → validation → returning data/errors from Server Actions.



yes....and also difference between onsubmit and action in forms....

Absolutely — this is **the key distinction** to understand before we move into

`useActionState` . 😊

`onSubmit` vs action in a `<form>`

They can both respond when a form is submitted, but they represent **two different approaches**.

1. Traditional React: `onSubmit`

You might write:

 </> TypeScript

```
"use client";

export default function Form() {
  async function handleSubmit(event: React.FormEvent) {
    event.preventDefault();

    const formData = new FormData(event.currentTarget);

    const response = await fetch("/api/analyze", {
      method: "POST",
      body: formData,
    });

    // handle response...
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="url" />
      <button type="submit">Analyze</button>
    </form>
  );
}
```

The flow is:



```
User clicks Analyze
      ↓
browser submits form
      ↓
onSubmit()
      ↓
JavaScript runs in browser
      ↓
event.preventDefault()
      ↓
fetch()
      ↓
Route Handler
      ↓
PageSpeed API
```

The important thing:

`onSubmit` gives you JavaScript control over the submission.

You decide:

Should I prevent the default?
Should I fetch?
Where should I fetch?
What should I send?
How should I process the response?
What should I show?



It's very flexible.

2. Server Actions: `action`

With a Server Action:

```
</> TypeScript

async function analyzeWebsite(formData: FormData) {
  "use server";

  const url = formData.get("url");

  // Call PageSpeed API
}
```



Then:

```
</> TypeScript

<form action={analyzeWebsite}>
  <input name="url" />
  <button type="submit">Analyze</button>
</form>
```



The flow becomes:

```
User clicks Analyze
  ↓
<form action={analyzeWebsite}>
  ↓
Next/React invokes Server Action
  ↓
Server
  ↓
PageSpeed API
```



You don't manually write:

```
</> TypeScript
```



```
event.preventDefault()
```

and you don't manually write:

```
</> TypeScript
```



```
fetch(...)
```

for the action invocation.

The framework handles that plumbing.

The simplest mental model

Think of it this way:

```
onSubmit
```

"When the form submits, give me the event and I'll handle everything."

```
</> TypeScript
```



```
<form onSubmit={handleSubmit}>
```

You take control.

```
action
```

"When the form submits, execute this action."

```
</> TypeScript
```



```
<form action={analyzeWebsite}>
```

You give the framework the function to execute.

But here's something REALLY important

Don't think:

```
onSubmit = client
action    = server
```



That's **too simplistic**.

`action` can be used with Server Actions, but `action` itself is fundamentally a **form mechanism**.

For example, an ordinary HTML form can have:

 HTML



```
<form action="/login" method="POST">
```

Here `action` means:

"Submit this form to this URL."

With Next.js/React Server Actions, you can instead provide a function:

 TypeScript



```
<form action={login}>
```

where `login` is a Server Action.

So there are actually two concepts hiding behind the same word:

HTML form action



URL to submit to

React form action



Function to execute



Server Action



Let's compare them directly

	<code>onSubmit</code>	<code>action</code> + Server Action
What is it?	Event handler	Form action

	onSubmit	action + Server Action
Function runs primarily where?	Browser	Server Action runs on server
Need "use client" ?	Usually yes	Not necessarily
Manually call fetch() ?	Usually	No
event.preventDefault() ?	Usually	No
Great for custom client behavior?	✅ Yes	Less directly
Great for server mutations?	Possible	✅ Excellent
Browser event available?	✅ Yes	❌ No
Receives FormData automatically?	No — you create it	✅ Yes
Can call secret server APIs safely?	❌ Not directly	✅

Your Performance Analyzer example

Suppose:

⌄ TypeScript

```
<input name="url" />
```

With onSubmit

```

Browser
  |
  | onSubmit
  ▼
handleSubmit()
  |
  | fetch()
  ▼
Route Handler
  |

```

▼
PageSpeed API

You are manually orchestrating the communication.

With Server Action

Browser
|
| form submission
▼
Server Action
|
| FormData
▼
PageSpeed API



Next/React handles the action invocation mechanism.

🤔 So why would we ever use `onSubmit` ?

Because sometimes you want **browser-side control**.

For example:

```
</> TypeScript  
  
<form onSubmit={handleSubmit}>
```



You might want to:

- manipulate the event
- perform client-side logic
- interact with browser APIs
- control exactly when/how a request happens
- use a client-side data-fetching library
- implement custom submission behavior

Whereas:

```
</> TypeScript  
  
<form action={analyzeWebsite}>
```



is fantastic when your intention is basically:

"User submitted this form → perform this server-side mutation."

That's exactly the sweet spot for Server Actions.



One more subtle but important thing 🚩

You can actually use **both**:

`</> TypeScript`



```
<form
  action={analyzeWebsite}
  onSubmit={handleSubmit}
>
```

They aren't mutually exclusive.

But now you're combining:

```
onSubmit
  ↓
client-side event behavior

AND

action
  ↓
form action / Server Action
```



You'd do this only when you genuinely need both behaviors. Don't add `onSubmit` just because it feels familiar.

The analogy I want you to remember 📖

Think about our earlier **doors** analogy:

`onSubmit`

```
Form
  ↓
"Hey browser, let my JavaScript handle this."
  ↓
onSubmit()
```



```
↓  
fetch()  
↓  
HTTP door 🚪
```

Server Action

```
Form  
↓  
"Execute this server function."  
↓  
Server Action door 🚪  
↓  
Server
```



So:

```
onSubmit = "I'll handle the submission."
```

```
action = "Here's what should happen when this form is submitted."
```

And **Server Actions** make that `action` a server-side function.

Next piece: `useActionState`

Now we can answer the really practical questions:

```
How do I show "Analyzing..."?  
How do I disable the button?  
How do I show validation errors?  
How does the Server Action return a result?  
How does the client receive that result?
```



That's where `useActionState` comes in. 🔥



YES! 😊 You found the exact point that usually causes confusion with `useActionState`.

Your understanding is actually **almost there**. The missing piece is what `formAction` really is.

Think of `formAction` as a wrapped version of `analyzeWebsite`

Start with your original Server Action:

```
</> TypeScript   
  
async function analyzeWebsite(previousState, formData) {  
  // ...  
}
```

Then we do:

```
</> TypeScript   
  
const [state, formAction] = useActionState(  
  analyzeWebsite,  
  initialState  
);
```

Conceptually, React does something like:

```
analyzeWebsite  
  ↓  
  wrapped  
  ↓  
formAction 
```

So `formAction` isn't some unrelated function.

It is essentially:

"A function that knows to invoke `analyzeWebsite` and also manage its state."

Then the form triggers `formAction`

We write:

```
</> TypeScript   
  
<form action={formAction}>
```

When the user clicks:

```
[ Analyze ] 
```

the form submission triggers:

```
formAction(formData)
```



And because `formAction` was created from:

```
</> TypeScript
```



```
useActionState(analyzeWebsite, initialState)
```

it knows:

```
"Ah, I need to invoke analyzeWebsite."
```



So the actual conceptual flow is:

```
User clicks Analyze
      ↓
<form action={formAction}>
      ↓
formAction(formData)
      ↓
analyzeWebsite(previousState, formData)
      ↓
Server Action executes
      ↓
returns new state
      ↓
useActionState updates `state`
      ↓
UI re-renders
```



🔥 That's the missing connection.

Let's make it super concrete

Suppose:

```
</> TypeScript
```



```
const [state, formAction] = useActionState(
  analyzeWebsite,
  initialState
);
```

You can mentally imagine:

</> TypeScript



```
formAction = (formData) => {  
  // React internally handles this  
  // and eventually invokes:  
  
  analyzeWebsite(state, formData);  
};
```

That's conceptual pseudocode, not the actual implementation.

Then:

</> TypeScript



```
<form action={formAction}>
```

means:

"When this form submits, call `formAction`."

And `formAction` means:

"Take this submission and run the Server Action I was created from."

Why doesn't the form just use `analyzeWebsite` directly?

Because we need something in between.

Without `useActionState`:

```
<form action={analyzeWebsite}>  
  ↓  
  analyzeWebsite()
```



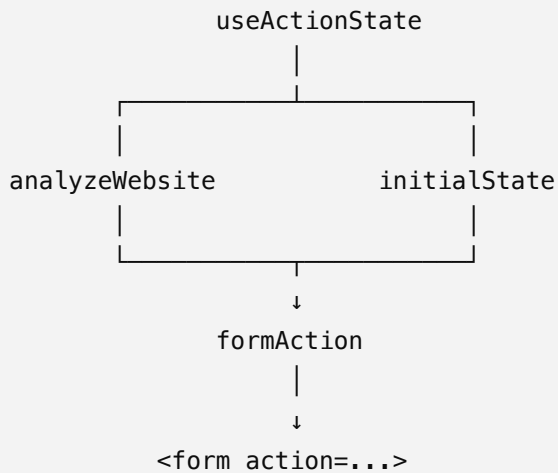
Simple.

But now we want React to also manage:

```
previous state  
  +  
form data  
  +  
server action result  
  ↓  
new state
```



So:



`formAction` is basically the **bridge** between the form and your Server Action.

One subtle thing you should notice 🧐

Your original action:

`</> TypeScript`

```
async function analyzeWebsite(  
  previousState,  
  formData  
)
```

is **not** what the browser directly calls.

The form calls:

`</> TypeScript`

```
formAction
```

and React's action-state machinery uses that to invoke:

`</> TypeScript`

```
analyzeWebsite(previousState, formData)
```

That's why the function signature changes when using `useActionState`.

Without `useActionState`, you might have:

```
</> TypeScript
```



```
async function analyzeWebsite(formData) {  
}
```

With `useActionState`:

```
</> TypeScript
```



```
async function analyzeWebsite(previousState, formData) {  
}
```

because React is now maintaining that state for you.



Your exact sentence, corrected

You said:

" `formAction` is like output from `useActionState` which takes `analyzeWebsite` and `initialState` as input but it gets triggered"

YES. Exactly.

Just add one missing sentence:

`formAction` is the function that the form triggers, and `useActionState` has connected that function to `analyzeWebsite`.

So:

```
useActionState(  
  analyzeWebsite,  
  initialState  
)  
  ↓  
[state, formAction]  
  ↓  
form submits  
  ↓  
formAction()  
  ↓  
analyzeWebsite()  
  ↓  
new state  
  ↓  
state
```



↓
UI

That's the whole mechanism. 🎯

And this is also why I said earlier **"don't directly put `analyzeWebsite` in the form"** — because once `useActionState` is involved, `formAction` is the action you give to the form.

Next, the really interesting question is: **while `formAction` is executing and `analyzeWebsite` hasn't returned yet, how does the button know to show `Analyzing...` ?** That's exactly where `useFormStatus` comes in.



yes and `formAction` is the function that the form triggers, and `useActionState` has connected that function to `analyzeWebsite`. but `formAction` also comes in output of `useActionState` right along with state ???

YES! 🎯 Exactly. You are saying it correctly.

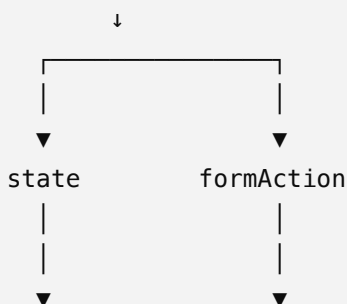
`formAction` is one of the values returned by `useActionState`, right alongside `state`.

`</>` TypeScript

```
const [state, formAction] = useActionState(
  analyzeWebsite,
  initialState
);
```

So the return value is:

`useActionState(...)`



"What is the current result?" "What should the form trigger?"

And then:

</> TypeScript

```
<form action={formAction}>
```

means:

```

Form submitted
  ↓
formAction()      ← returned by useActionState
  ↓
analyzeWebsite()  ← the action you originally passed in
  ↓
returns new state
  ↓
state gets updated
  ↓
UI re-renders
  
```

So you can think of `useActionState` as taking:

```

analyzeWebsite + initialState
  ↓
useActionState
  ↓
├── state
└── formAction
  
```

`state` = the result/state you read.

`formAction` = the action function you give to `<form>` .

And `formAction` is connected internally to the `analyzeWebsite` you passed in.

So your understanding is now **spot on**. 🎯

One tiny terminology point: `formAction` isn't the *result* of the Server Action. It's a **function returned by `useActionState` that wraps/connects the Server Action.**

That's the distinction.

Now we can move to `useFormStatus` → **pending/loading state**. 🚀



its confusing ..see formAction is triggered by form when submission but it is the output returned from useActionState which takes inputs initial state and analyze website but u r also saying formaction invokes analyze website which is input but formaction is output from useActionstate..total confusion

YES — I see exactly why this feels contradictory. 🤔 The problem is that we're mixing up **"who creates the function"** and **"who calls the function."**

Let's completely reset and use a very simple analogy.

Forget Server Actions for 1 minute

Imagine this normal JavaScript:

`</> TypeScript`

```
function sayHello(name) {  
  console.log("Hello", name);  
}  
  
function createButton(callback) {  
  return function buttonClick(value) {  
    callback(value);  
  };  
}  
  
const button = createButton(sayHello);
```

Here:

```
createButton(...)  
  ↓  
returns a NEW function  
  ↓  
button
```

So:

```
sayHello = INPUT to createButton
```

```
button = OUTPUT from createButton
```

But what happens when we do:

```
</> TypeScript  
button("Gopi");
```

?

The returned `button` function internally calls the original:

```
</> TypeScript  
sayHello("Gopi");
```

So there is **NO** contradiction.

A function can:

1. be returned from another function
2. later call the function that was passed into the original function

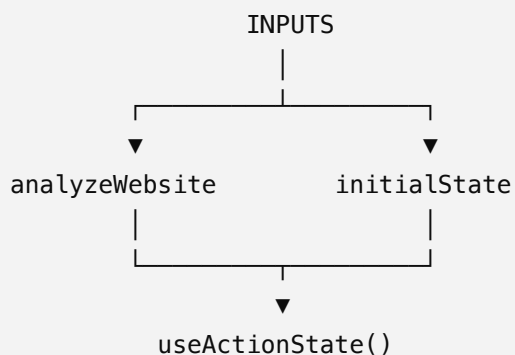
That's exactly what is happening with `useActionState`.

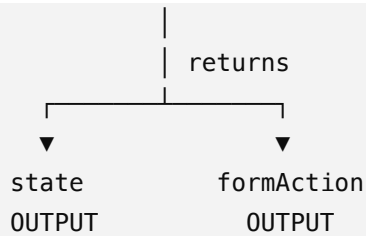
Now come back to `useActionState`

We have:

```
</> TypeScript  
const [state, formAction] = useActionState(  
  analyzeWebsite,  
  initialState  
);
```

There are two inputs:





Yes. `formAction` is absolutely an output. ✅

You are 100% right about that.

So what happens when the form submits?

We have:

`</>` TypeScript

```
<form action={formAction}>
```

The form says:

"When you submit, I will call this `formAction` function."

So:

form submission

↓

`formAction()` ← OUTPUT from `useActionState`

Now here's the crucial part:

What did `useActionState` create that `formAction` to do?

It created `formAction` as a wrapper around:

`analyzeWebsite`

So internally, conceptually:

`formAction(formData)`

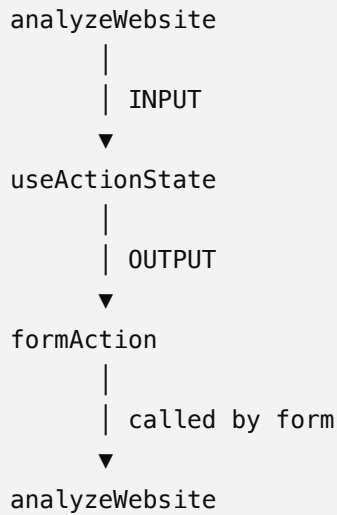
↓

`useActionState's internal logic`

↓

`analyzeWebsite(previousState, formData)`

Therefore:



🔥 The key sentence

Read this slowly:

`analyzeWebsite` is the input that tells `useActionState` **WHAT** server action to manage. `formAction` is the output function that tells the form **HOW** to trigger that managed action.

That's it.

Think of a remote control 📺

Suppose I give a TV system this:

TV
+
remote-control configuration

and it creates a remote:

createRemote(TV)
↓
remote

The remote is **created as an output**.

But when you press the remote button:

remote.power()
↓

```
TV.power()
```

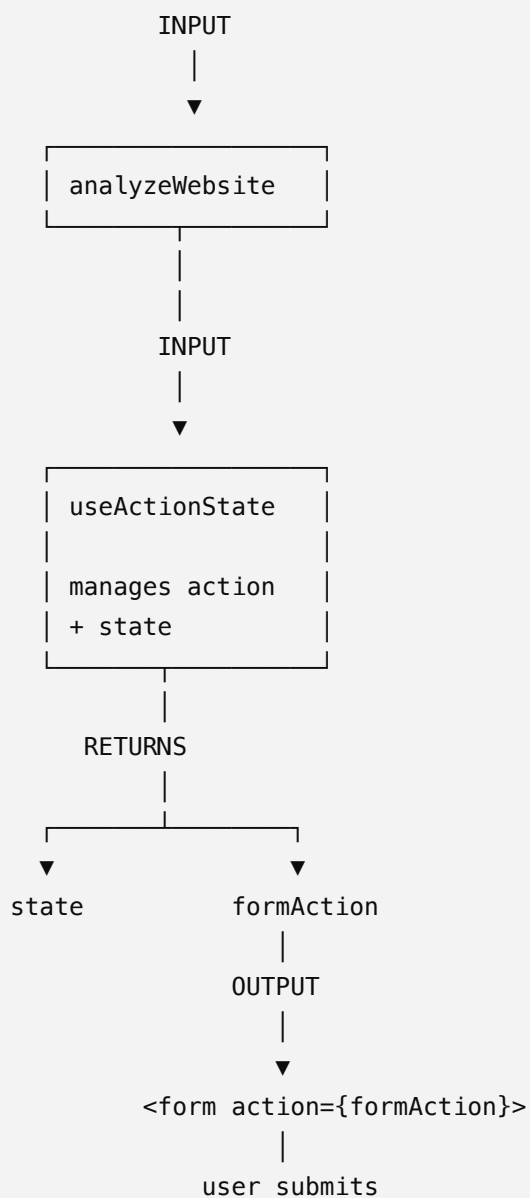
The fact that `remote` was an output doesn't mean it can't operate the thing that was supplied as input.

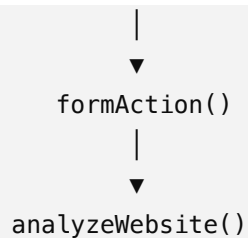
Same idea:

```
analyzeWebsite
  ↓
useActionState
  ↓
formAction
  ↓
analyzeWebsite
```



Let's draw the whole thing with arrows labeled





Notice the important thing:

The bottom arrow going from `formAction` to `analyzeWebsite` does NOT mean `analyzeWebsite` is an output.

It means:

the returned `formAction` uses/calls the action that was provided as input.

One last distinction

There are two completely different relationships here:

Relationship 1 — Creation

```
useActionState(  
  analyzeWebsite,  
  initialState  
)  
  ↓  
returns  
  ↓  
formAction
```



Relationship 2 — Execution

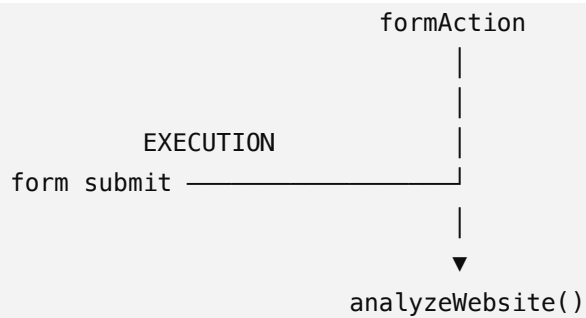
```
form submits  
  ↓  
formAction()  
  ↓  
analyzeWebsite()
```



Put those together:

```
      CREATION  
analyzeWebsite → useActionState  
                  ↓
```





That's the piece that was getting tangled. ❤️

So yes: `formAction` is an output of `useActionState` . Absolutely.

And that returned function is designed to invoke/manage the `analyzeWebsite` function that you originally supplied.

Now the next piece, `useFormStatus` , will be much easier because we'll only add **one more thing**: "is that returned `formAction` currently running?"



ChatGPT can make mistakes. Check important info.